Real-Time Guaranteed Monitoring for a Drone Using Interval Analysis and Signal Temporal Logic

Antoine Besset*, Julien Alexandre dit Sandretto* and Joris Tillet*

***Abstract*— This paper presents a guaranteed model-based approach for monitoring drone trajectory, providing real-time guarantees with a simplified dynamic model. ROS components are introduced for real-time implementation, enabling monitoring and adjustments in both simulations and actual systems. We extend the application of set-based simulation by formalizing timing conditions with Signal Temporal Logic (STL) and incorporating Boolean interval arithmetic to handle undetermined behaviors. The method compares model-based fault prediction using a stochastic approach with a set-based method, which manages bounded uncertainties and offers guarantees. Experimental validation, including comparisons against Monte Carlo methods, demonstrates the approach ability to ensure safety in worst-case scenarios while remaining suitable for real-time processing.**

I. Introduction

Ensuring system reliability and safety is critical in robotics. Despite careful design, various factors such as model inaccuracies, sensor noise, and environmental changes can lead to failures. Monitoring helps detect these issues early, providing safeguards that maintain reliability and safety. Model-based methods assists by forecasting future system states and identifying discrepancies between nominal and predicted behaviors [1], [2]. However, accurately modeling systems such as quadrotors is challenging due to their highly nonlinear dynamics and aerodynamic interactions [3], [4].

A common approach to handling uncertainties in system verification is the use of stochastic methods [5], [6]. However, in safety-critical systems, guarantees are required. This is why we prefer interval analysis, which has already been applied in robotics [7], [8]. These methods are effective for real-time monitoring by verifying whether the system remains within specified bounds over time [9]. Beyond spatial constraints, many practical requirements involve timing conditions. For instance, a drone may need to maintain a safe distance for several seconds before reaching a checkpoint. Signal Temporal Logic (STL) [10] enables the specification of both timing constraints and system variable values. This formalism offers expressiveness in temporal conditions and eliminates ambiguity. Previous studies have used STL for monitoring with precomputed reachable sets or static verification [11], [12], and similar approaches exist for Linear Temporal Logic (LTL) [13] in real-time settings. LTL operates on discretized states as propositions and discretized time, whereas our approach focuses on continuous behavior through STL. Other works investigate the quantitative semantics of STL formulas, assessing how well a signal satisfies or violates temporal constraints [14], [2], which is not our goal in a guaranteed context.

In this paper, we integrate guaranteed simulation with STL to develop a real-time monitoring method. We employ set-based numerical integration [15], [16] to generate "tubes" that encapsulate all possible system evolutions under bounded uncertainties. The studied system is modeled as a switched system, enabling the integration of control into the predicted behavior. We then verify these tubes against STL specifications. By operating on sets instead of single trajectories, our method accommodates simplified or partially known models and refines predictions using known control actions.

Our contributions are twofold: first, we propose a real-time monitoring approach capable of handling uncertain models and timing requirements. Second, we implement this approach as a Robot Operating System (ROS) node, facilitating deployment in both simulations and robots. Finally, we compare the computational effort of the proposed approach with a Monte Carlo method, demonstrating that formal set-based monitoring can achieve safety guarantees within practical time constraints.

II. Preliminaries

A. Interval Analysis

The main underlying tool used is interval analysis [17]. An interval is a mathematical concept that considers a set of values. An interval $[x] = [x_{\min}, x_{\max}]$ defines the set of real numbers x such that $x_{\min} \leq x \leq x_{\max}$. The set of intervals is denoted by $\mathbb{IR}$. Interval arithmetic extends the elementary operations defined in $\mathbb{R}$ to the set of intervals $\mathbb{IR}$. We use the notation $[\mathbf{x}]$ to denote a vector of intervals, which is equivalent to a Cartesian product of intervals, also known as a box. In this article, interval analysis is employed to account for all uncertainties in the system. It encompasses the full range of possible values for the uncertainties and, therefore, the worst-case scenario [7].

To perform guaranteed integration of differential equations, we use a tool based on interval arithmetic and Runge-Kutta methods called DynIbex[1]. It is designed for the analysis and verification of dynamical systems, particularly those described by Ordinary Differential Equations (ODEs). To ensure that the numerical solution obtained encloses all possible solutions of the ODEs, DynIbex employs validated numerical integration methods [15]. This means that each approximate solution obtained through numerical integration

*U2IS, ENSTA, Institut Polytechnique de Paris, Palaiseau, France

[1] https://perso.ensta-paris.fr/~chapoutot/dynibex/

is an over-approximation that includes all possible solutions of the system. The dynamical functions f, used to define the systems as $\dot{y}(t) = f(t, y(t))$, must be continuous, with f being globally Lipschitz with respect to y, to ensure convergence and accuracy in the simulations. Other tools, such as CORA[2], used in [12], and CAPD[3] [18], enable the reachability analysis of continuous dynamic systems.

In solving an Initial Value Problem (IVP) for a differential equation representing the system dynamics, we compute an enclosure $[\tilde{y}_j]$ of the solution for each time interval $[t_j, t_{j+1}]$, where $t_{j+1} = t_j + h_j$ and h_j is the integration step size. This enclosure satisfies:

$$y(t, [y_j]) \subseteq [\tilde{y}_j],\ \forall t \in [t_j, t_{j+1}] \text{ with } \bigcup_{j=0}^{N-1} [t_j, t_{j+1}] = [t_0, T], \quad (1)$$

where t_0 is the simulation start time and T is the end time.

In this paper, we refer to the set of solutions of this ODE as a *tube*, denoted by $[\tilde{\mathbf{y}}](t)$, $t \in [t_0, T]$. Unlike reachability analysis using Lyapunov functions, this tube represents successive reachable sets over a finite time horizon, allowing direct access to the reachable set at a given time ($< T$).

B. Signal Temporal Logic

To encompass temporal constraints in monitoring, STL is used [10], [19]. It is an extension of LTL designed to specify properties of continuous-time signals. STL is adapted to handle systems where state variables evolve continuously, such as in robotics and control systems.

In the context of monitoring, STL formulas enable the expression of temporal properties in terms of time bounds, combining logical operators with bounded temporal operators. For example, an STL formula could express that "if a certain condition (predicate) is true at a given time, then another condition must be true within a specific time interval." The syntax of STL is defined recursively as follows:

$$\varphi := \mu \mid T \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid \varphi_1 U_{[a,b]}\, \varphi_2, \quad (2)$$

where φ is an STL formula, T is the tautology, and μ represents an atomic predicate, which, in the context of verification, maps a continuous domain (e.g., $\mathbb{R}^m$) to a Boolean value, thereby allowing the specification of conditions on signals (e.g., $x > 3$) [10]. Finally, $U_{[a,b]}$ is the "Until" operator bounded by the time interval $[a,b]$. Derived operators such as $G_{[a,b]}$ and $F_{[a,b]}$ are also defined to express additional temporal constraints [10]. The semantics are as follows:

$$\begin{aligned}
(s,t) \models \mu &\iff \pi_\mu(s)[t] = T \\
(s,t) \models \neg\varphi &\iff (s,t) \not\models \varphi \\
(s,t) \models \varphi_1 \vee \varphi_2 &\iff (s,t) \models \varphi_1 \text{ or } (s,t) \models \varphi_2 \\
(s,t) \models \varphi_1 U_{[a,b]} \varphi_2 &\iff \exists t' \in [t+a, t+b],\ (s,t') \models \varphi_2 \\
&\qquad \text{and } \forall t'' \in [t,t'],\ (s,t'') \models \varphi_1 \\
F_{[a,b]}\varphi = TU_{[a,b]}\varphi, &\quad G_{[a,b]}\varphi = \neg(F_{[a,b]}\neg\varphi).
\end{aligned} \quad (3)$$

where (s,t) represents the trace of a signal at time t, and $\pi_\mu(s)$ is a test function that evaluates a given predicate on the signal. The "Finally" operator $F_{[a,b]}$ expresses the existence of a satisfaction within a time interval, while the "Globally" operator $G_{[a,b]}$ specifies that the subformula must hold throughout the entire time interval.

C. Boolean intervals

Considering sets, constraints can be satisfied for some values and violated for others, resulting in an outcome that is neither strictly true nor false. To handle this, we use Boolean intervals, which extend traditional Boolean logic to manage uncertainty in set-based contexts [20]. A Boolean interval is a subset of $\mathbb{B}$, belonging to the interval Boolean set $\mathbb{IB} = \{\emptyset, [0,0], [1,1], [0,1]\}$. Here, the values $\emptyset$ and $[0,1]$ represent, respectively, impossible and undetermined states. The logical operations on Boolean values are extended to Boolean intervals $[a]$ and $[b]$ as follows:

$$\begin{aligned}
[a] \wedge [b] &= \{a \wedge b \mid a \in [a], b \in [b]\}, \\
[a] \vee [b] &= \{a \vee b \mid a \in [a], b \in [b]\}, \\
\neg[a] &= \{\neg a \mid a \in [a]\}.
\end{aligned}$$

For example, the behavior of an undetermined interval $[0,1]$ is as follows [20]:

$$\begin{aligned}
0 \wedge [0,1] = 0, &\quad 0 \vee [0,1] = [0,1], \\
1 \wedge [0,1] = [0,1], &\quad 1 \vee [0,1] = 1.
\end{aligned}$$

We extend the test on predicate μ defined on a signal with this concept. For example, when verifying a predicate and checking an STL formula, if at time $t \in [t_j, t_{j+1}]$ the solution $[\tilde{y}_j]$ cannot determine whether the predicate is satisfied, the result is undetermined, i.e., $[0,1]$.

III. Monitored system: hypotheses and problem formulation

This section presents the problem formulation for model-based monitoring of a dynamic system, detailing the representation of the drone model and the handling of bounded uncertainties.

A. System model

In this model-based monitoring approach, we focus on studying the behavior of our system over a finite time horizon h. In robotics, the dynamical behavior of a system is typically represented using state-space models [21]:

$$\dot{\mathbf{y}}(t) = \mathbf{f}(\mathbf{y}(t)) \quad \text{with} \quad \mathbf{y}(\mathbf{0}) \in [\mathbf{y_0}] \quad \text{and} \quad t \in [0,h], \quad (4)$$

where $\mathbf{y}$ represents the state vector of the system. The function $\mathbf{f}$ encapsulates the system dynamics. The control input must be incorporated into the simulation to accurately predict the system's behavior. Additionally, the command must be differentiable to solve the ODE representing the dynamic system. We represent the system as a switched system, composed of multiple subsystems, each operating under a distinct mode. The system transitions between these modes based on a specified switching rule. We focus on sampled switched systems [16], where switching occurs at regular instants. For example, a drone's planned trajectory is

[2] https://tumcps.github.io/CORA/ [3] http://capd.ii.uj.edu.pl/

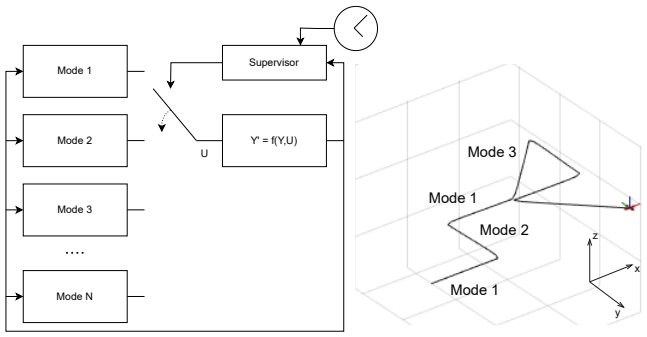

Fig. 1. Trajectory planned as a sequence of control modes.

represented by multiple control modes, as shown in Fig. 1. Given a sampling period $\tau > 0$, the system switches modes at times $\tau, 2\tau, 3\tau, \ldots$. If the sampling period is $\tau = 1s$, the system might follow a switching sequence $\pi = (1,2,1,3)$, operating in mode 1 during $[0s, 1s[$, switching to mode 2 during $[1s, 2s[$, returning to mode 1 during $[2s, 3s[$, and then switching to mode 3 during $[3s, 4s[$. The representation of the monitored system is described as an ODE:

$$\dot{\mathbf{y}}(t) = f_{\sigma(t)}(\mathbf{y}(t)), \tag{5}$$

where $\mathbf{y} \in (\mathbb{R}^+ \to \mathbb{R}^n)$ represents the evolution of the system, and $\sigma(\cdot) : \mathbb{R}^+ \to U$ is the switching rule that maps time to the set $U = \{1, \ldots, N\}$ of possible modes.

The periodic nature of the switching helps to simulate on a fixed simulation time horizon, ensuring predictable transitions between modes, e.g., $h = N \times \tau$. In the case of trajectory planning, the goal is to choose $\sigma(t)$ such that the overall system meets the desired objectives over the entire time horizon, as illustrated in Fig. 1. In practice, during simulation, this sequence is simply a series of ODEs to solve, where the final state of each solution serves as the initial condition for the next mode. Finally, in actual drone path planning, trajectories follow the same switching rule $\sigma(t)$ as in the system model.

B. Guaranteed Simulation

Introduced at the beginning of this section, our approach is to use the dynamical model to predict the future state within a given time horizon. The predicted trajectory $\mathbf{y}$ is evaluated to determine if a fault will occur [1]. To ensure guarantees in model-based monitoring, it is crucial to account for all uncertainties. Thus, we employ *guaranteed simulation*, which computes an enclosure of the solution while explicitly considering these uncertainties. The uncertainties are represented as bounded sets [7]. The set of uncertain parameters is denoted as $[p] \in \mathbb{IR}^n$. Following the previous modeling approach, the system is represented as a differential inclusion:

$$\dot{\mathbf{y}}(t) \in f_{\sigma(t)}(\mathbf{y}(t), [p]), \tag{6}$$

Modes in switched systems can be nonlinear, which can also result in a nonlinear relationship between parameters and states. Although the following experiment utilizes a linear system representation, the proposed approach extends to nonlinear systems, as the set-based simulation framework employed in this work is capable of handling nonlinear representations.

C. Predicate evaluation

To ensure that the system adheres to the expected behavior, we define these expectations using an STL formula. If the predicted states violate this formula, the monitor identifies a potential fault. In a monitoring approach, defining the formula requires specifying predicates on a given signal [10], which, in our case, corresponds to a trajectory. However, in this paper, predicates are evaluated over the enclosure of possible trajectories. Therefore, we express the trace of a tube as:

$$[\varepsilon_{\tilde{\mathbf{y}}}] = \{[\tilde{\mathbf{y}}](t),\, t \in [0,h]\}. \tag{7}$$

To account for satisfaction over this enclosure, we employ a Boolean interval approach, as detailed in Section I.C. In the following section, we will discuss the handling of these predicates in greater detail.

IV. Monitor principle

Building on the problem formulation, the following section presents the implementation of fault detection on tubes.

A. Fault detection

The monitoring process aims to recognize if faults occur [1]. In our case, this is done by verifying an STL formula. We will provide an example that will also be used in Section V. Consider the following specification: "The drone must avoid obstacles within a given time horizon and remain inside a designated area." The corresponding STL formula is given by:

$$\varphi_{\text{flag}} = G_{[0,N\times\tau]} \neg \mu_{collision} \wedge G_{[0,N\times\tau]} \mu_{env}. \tag{8}$$

To verify the satisfaction of a formula from punctual trajectories to set-based representations, we introduce two set predicates, μ_i and μ_d. The inclusion predicate μ_i is defined as $\mathbf{y}(t) \in \mathscr{X}^{\mu}$, while the disjointness predicate μ_d is given by $\mathbf{y}(t) \notin \mathscr{X}^{\mu}$. Here, $\mathscr{X}^{\mu} \in \mathbb{IR}^n$ is a set in an n-dimensional space.

For example, a drone trajectory is represented by its trace $\varepsilon_{\mathbf{y}}$. In the STL formula, the predicate $\mu_{collision}$ is defined as $\mathbf{y}(t) \in [\mathbf{o}]$, where $[\mathbf{o}] \in \mathbb{IR}^3$ is a fixed obstacle, while μ_{env} is given by $\mathbf{y}(t) \in [\mathbf{S}]$, where $[\mathbf{S}] \in \mathbb{IR}^3$ denotes the flying zone. Extending these predicates to a set-based context, we define the satisfaction as follows.

For the inclusion predicate:

$$([\varepsilon_{\tilde{\mathbf{y}}}], t) \vDash \mu_i \equiv \begin{cases} 1, & \text{if } [\tilde{\mathbf{y}}](t) \subseteq \mathscr{X}^{\mu}, \\ 0, & \text{if } [\tilde{\mathbf{y}}](t) \cap \mathscr{X}^{\mu} = \emptyset, \\ [0,1], & \text{otherwise.} \end{cases} \tag{9}$$

For the disjointness predicate:

$$([\varepsilon_{\tilde{\mathbf{y}}}], t) \vDash \mu_d \equiv \begin{cases} 1, & \text{if } [\tilde{\mathbf{y}}](t) \cap \mathscr{X}^{\mu} = \emptyset, \\ 0, & \text{if } [\tilde{\mathbf{y}}](t) \subseteq \mathscr{X}^{\mu}, \\ [0,1], & \text{otherwise.} \end{cases} \tag{10}$$

In practice, the predicates used are often redundant. At the formula level, we establish the following equivalence:

$$([\varepsilon_{\tilde{\mathbf{y}}}],t) \vDash \neg\mu_i \equiv ([\varepsilon_{\tilde{\mathbf{y}}}],t) \vDash \mu_d. \tag{11}$$

This choice ensures guarantees in our approach: either all possible positions within the enclosure $[\tilde{\mathbf{y}}](t)$ satisfy the punctual predicate, i.e., $\mathbf{y}(t) \in \mathscr{X}^{\mu}$, or none of them do, i.e., $\mathbf{y}(t) \notin \mathscr{X}^{\mu}$.

Furthermore, set predicates can represent classical scalar predicates, such as $y \geq 3$ for $y \in \mathbb{R}$, using level sets. For example, this condition can be rewritten as $y \in [3,\infty[$, which is naturally representable within our framework.

Since DynIbex version 2.3, temporal operators such as $U_{[a,b]}$, $G_{[a,b]}$, and $F_{[a,b]}$ are available. Conjunction and disjunction of these temporal operators, along with predicate tests, are also supported. The available extension to STL formula verification in this implementation is given by:

$$([\varepsilon_{\tilde{\mathbf{y}}}],t_s) \models \bigwedge_{n=1}^{N} \bigvee_{k=1}^{K} \varphi_{nk}, \tag{12}$$

where each φ_{nk} is recursively defined as follows:

$$\varphi_{nk} ::= \mu_i \mid T \mid \neg\varphi \mid \varphi_1 \vee \varphi_2 \mid G_{[a,b]}\varphi_1 \mid F_{[a,b]}\varphi_1 \mid \varphi_1 U_{[a,b]}\varphi_2$$

Additionally, the initial simulation time, denoted as t_s, is determined by the calling frequency of the monitor. An example of an STL formula that can be handled using DynIbex is: $\varphi_{\text{flag}_2} = (G_{[0,N\times\tau]}(\neg\mu_c \wedge (\mu_p \Rightarrow F_{[c,d]}\mu_f))) \wedge (\mu_e U_{[a,b]}\mu_p)$. This states that the drone must avoid obstacles while remaining within the designated flight zone until it reaches a checkpoint within the time interval $[a,b]$. Furthermore, if the drone reaches the waypoint p, it must proceed to park in the final zone f within the time interval $[c,d]$.

If we decompose the predicate $\mu_{collision}$ in sub-STL formula [10], across K obstacles, the formula becomes:

$$G_{[0,N\times t]}\neg\mu_c = \bigwedge_{j=1}^{K} G_{[0,N\times t]}\neg\mu_{c_j}. \tag{13}$$

After computing the guaranteed simulation, we evaluate the formula at the simulation start time t_s: $([\varepsilon_{\tilde{\mathbf{y}}}],t_s) \vDash \varphi_{\text{flag}}$, using the tube computed over the horizon h. In case of undetermined status, it propagates within the entire STL formula, following the rules outlined in Section II, C.

Finally, we define a three-state flag $\{0,1,[0,1]\}$: the tube either guarantees compliance with formula (1), guarantees non-compliance (0), or remains undetermined ([0, 1]).

B. Sliding horizon time

Since we are verifying formula satisfaction, we outline the simulation approach used in this monitoring process. The sliding horizon approach, illustrated in Fig. 2, continuously updates future state predictions by measuring the current state and simulating system behavior over fixed time intervals called "horizons." As the drone moves, simulations are performed at a fixed frequency, ensuring a continuous prediction of the system's future behavior for the next h seconds. This guarantees that at any moment, a forecast is available for the upcoming h seconds, enabling real-time monitoring based on the latest state estimates.

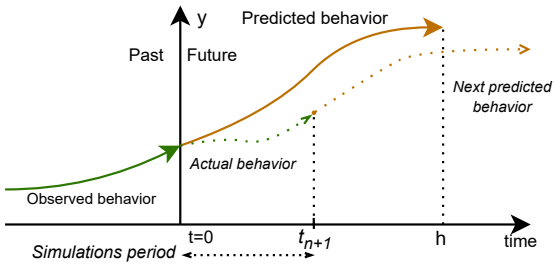

Fig. 2. Schematic of sliding horizon time

Practically, since the initial states remain unchanged during processing and between simulations, a tunneling effect occurs, meaning the actual evolution of the system is only captured at discrete points in time. Therefore, it is crucial that the simulation time horizon h is significantly longer than the period between simulations.

V. Experimentation

The experiment presents a reach-avoid problem with a predefined trajectory. This setup enables the evaluation of forward prediction while integrating control dynamics within the model. A simple STL formula is employed to validate the system behavior (Section IV, A).

A. Control based on motion primitives

The switched system integrates different control modes using motion primitives. Motion primitives are precomputed control actions that guide a robot movement, ensuring smooth transitions between trajectories while adhering to its dynamical constraints. In the experiment, trajectories consist of eight possible modes and are time-bounded within $[0,\tau]$, as illustrated in Fig. 3.

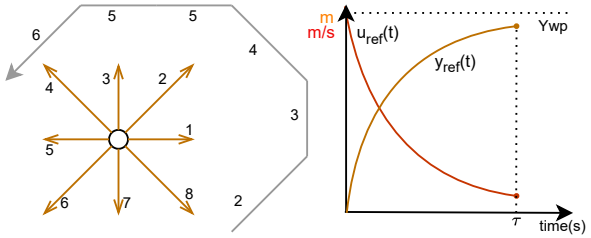

Fig. 3. Schematic of motion planning using motion primitives. Evolution on y axis.

For each pre-computed trajectory, the differential equation is a P-corrector based control loop from initial position to a waypoint. We use a first order dynamic model for the drone:

$$\dot{\mathbf{y}}_{\text{ref}} = \mathbf{u}_{\text{ref}} \text{ with } \mathbf{u}_{\text{ref}} = K_p \times (\mathbf{Y}_{\text{wp}} - \mathbf{y}_{\text{ref}}). \tag{14}$$

We denote $y_{\text{ref}}(t)$ and $u_{\text{ref}}(t)$ as the precomputed motion primitives as seen in Fig. 3. To track the given trajectory, the drone velocity control is:

$$\mathbf{u}(t) = K_{mp}(\mathbf{y}_{\text{ref}}(t) - y(t)) + \mathbf{u}_{\text{ref}}(t), \tag{15}$$

with K_{mp} being the gain of P corrector.

B. Order 1 and order 2 dynamic model in closed loop

The drone is assumed to be holonomic. We propose two distinct dynamical models for the monitoring. The first model is a system controlled by speed along each axis. The second model accounts for inertia, making it more accurate. These models are intentionally simplified compared to dynamical models that account for the rotation of the quadrotor [4]:

$$\dot{\mathbf{y}} = \mathbf{u}, \quad \text{order 1}; \quad \ddot{\mathbf{y}} = \frac{K}{M} \times (\mathbf{u} - \dot{\mathbf{y}}), \quad \text{order 2}. \tag{16}$$

M is the drone mass, K is the gain of an internal P corrector to reach the controlled speed u. In simulation, we keep an order 1 model on the z axis. The complete ODE for order 2 to solve, with command, is represented as:

$$\mathbf{y} = \begin{pmatrix} x \\ y \\ z \\ \dot{x} \\ \dot{y} \\ x_{\text{ref}} \\ y_{\text{ref}} \\ z_{\text{ref}} \end{pmatrix}, \dot{\mathbf{y}} = \begin{pmatrix} \dot{x} \\ \dot{y} \\ K_{mp}(z_{\text{ref}} - z) + K_p(Z_{\text{wp}} - z_{\text{ref}}) \\ \frac{K}{M}(K_{\text{mp}}(x_{\text{ref}} - x) + K_p(X_{\text{wp}} - x_{\text{ref}}) - \dot{x}) \\ \frac{K}{M}(K_{\text{mp}}(y_{\text{ref}} - y) + K_p(Y_{\text{wp}} - y_{\text{ref}}) - \dot{y}) \\ K_{\text{p}}(X_{\text{wp}} - x_{\text{ref}}) \\ K_{\text{p}}(Y_{\text{wp}} - y_{\text{ref}}) \\ K_{\text{p}}(Z_{\text{wp}} - z_{\text{ref}}) \end{pmatrix} \tag{17}$$

N.b: (x,y,z) are the cartesian coordinate of the drone.

C. Parameter identification

To account for the simplifications in the dynamical models, we will introduce interval parameters as in [9].

For the first-order model:

$$[\dot{\mathbf{y}}] = [\alpha_1] \cdot [\mathbf{u}] + [\boldsymbol{\beta_1}] \tag{18}$$

$[\alpha_1]$ adjusts the assumption of immediate command responses by incorporating the drone's speed profile and accounting for uncertainties related to forces that depend on velocity. $[\beta_1]$ compensates for uncertainties arising from perturbations and accounts for ambient noise and bias in the control inputs, including noise induced by the drone itself [9].

For the second-order model:

$$[\ddot{\mathbf{y}}] = [\alpha_2] \cdot \frac{K}{M} \times ([\mathbf{u}] - [\dot{\mathbf{y}}]) + [\boldsymbol{\beta_2}] \tag{19}$$

$[\alpha_2]$ adjusts the assumption of immediate force response by incorporating the drone's rotation profile to calculate the projected force on each axis and accounts for uncertainties related to rotation that depend on the target velocity. $[\beta_2]$ compensates for uncertainties arising from perturbations.

Furthermore, when incorporating sensor readings into the initial state of the ODE, $\mathbf{y_0}$ becomes $[\mathbf{y_0}]$ as bounded uncertainties are introduced in the measurement or state estimation due to filtering. For instance, the speed command could be used as the initial speed because it simplifies data acquisition.

The goal of parameter identification is to ensure that all drone trajectories remain within the simulated tube. In practice, command errors and perturbations are unknown but can be experimentally measured. By logging multiple trajectory runs, we quantify velocity and position errors, defining an enclosure around the velocity response α and adjusting the input parameter β to account for speed errors and disturbances. Reachset conformance approaches enable such identification [22]. Pre-tuning is performed using a realistic simulation model, followed by inclusion tests on multiple simulations and system logs to ensure the uncertainty model fully encloses observed positions. Table I lists an example of pre-tuned parameters, where "Ld" represents low-force disturbance and "Hd" represents high-force disturbance in the drone simulation.

TABLE I
VALUES FOR PARAMETERS.

	$[\alpha_1]$	$[\beta_1]$	$[\alpha_2]$	$[\beta_2]$
Ld	[0.95, 1.01]	[-0.08, 0.08]	[0.812, 1.125]	[-0.18, 0.18]
Hd	[0.95, 1.01]	[-0.36, 0.36]	[0.812, 1.125]	[-3.3, 3.3]

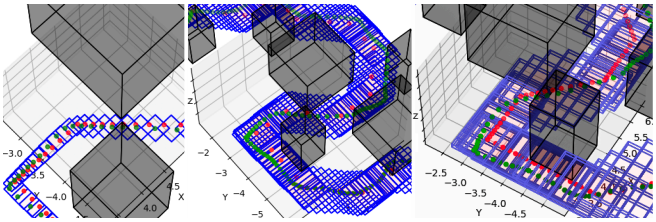

Fig. 4. Results of tubes with inclusion. The two left-side pictures are the second order (Ld and Hd as follows), and the right one is first order (Hd). Red dots are the planned trajectory, green ones are the actual trajectory, finally blue boxes are tubes representations.

As shown in Fig. 4, the first-order model exhibits larger uncertainty boxes along the x and y axes due to command variations, whereas the second-order model better accounts for inertia, resulting in offset but tighter enclosures. Since altitude remains constant, velocity commands are minimal, leading to flat boxes.

Over 20 simulated time horizons with varying obstacle configurations and low disturbances, the first-order model yielded three "MAYBE"-$[0,1]$ flags for collision uncertainty, whereas the second-order model consistently ensured collision-free performance. Nonetheless, the first-order model maintained safety guarantees in 17 out of 20 cases, demonstrating reliability despite its simplifications. If the system dynamics are not accurately described, the bounded input uncertainties β will be greater relative to the control input, leading to an expanding tube over time. This leads to a scenario where the minimum control input to counteract deviations is equals to the maximum bounded input uncertainty along the same axis. Specifically, for the first-order model, the relation $\overline{[\alpha_1] \cdot [u]} = \overline{[\beta_1]}$ holds at certain boundaries. This behavior is illustrated in Fig. 4 where the tube increases in both directions to mitigate inertial behavior. This results in growing conservatism, especially over long time horizons. To mitigate this effect, we employ a sliding horizon approach, where the position is regularly updated to maintain accuracy and reduce pessimism.

D. ROS implementation

The design of the monitor requires simulation for development and validation. To enable seamless switching between simulation and real experiments, the monitor is implemented as a ROS node within the Melodic distribution. The monitor node subscribes to topics where the system state properties are published, whether from the simulation or the actual experiment. Additionally, pre-tuning of interval parameters is necessary for effective deployment.

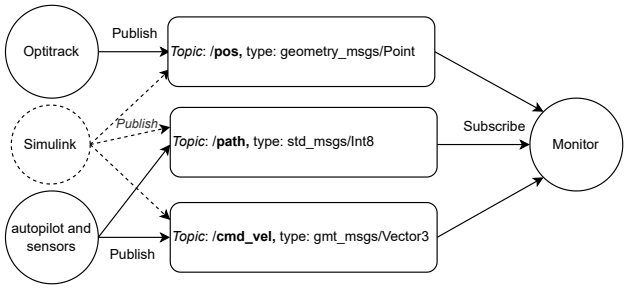

Fig. 5. ROS architecture. circles are nodes, rectangles are topics

Simulation is performed in Simulink, where states are published using the "ROS Publish" block. In real-world experiments, the drone's position is tracked with an OptiTrack system[4], achieving approximately 2 mm accuracy at up to 100 Hz [9]. The drone operates at 100 Hz using the DJI Tello node `tello_driver`[5], controlling speed along each axis. The monitor stores the full path, while only the current index is communicated via the `/path` topic (see Fig. 5).

E. Simulation test

To test the system before real-world experiments, we implemented it in a drone simulation, as detailed above. The Fig. 6 shows three possible states of the monitor using first-order dynamics with $h = 4s$. On the top, the computed tube goes out of the environment, which results in a "*NO*" flag. The second case shows a "*MAYBE*" flag indicating uncertainty about whether the drone will avoid an obstacle. Lastly for the "YES", the computed tube does not cross any obstacle and therefore guarantees that the drone will not collide with an obstacle.

F. Comparison with Monte-Carlo method and results

To benchmark our methods against a Monte Carlo approach, we employed the same first-order and second-order models. In both cases, uncertainties in the initial state and dynamical model were represented as uniform random variables with the same bounds as the set-based simulation, ensuring a consistent comparison between the stochastic and interval-based methods. In our approach, we implement a Monte Carlo method and monitor the variance of the point cloud during the simulation. The process is terminated once the variance stabilizes, i.e., when it reaches an asymptotic value. This adaptive stopping criterion ensures that the simulation runs efficiently while maintaining accuracy,

[4] https://optitrack.com/ [5] https://github.com/anqixu/tello_driver/

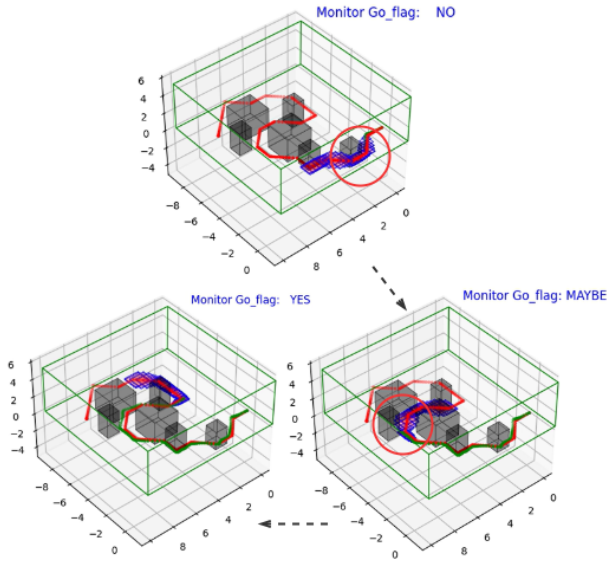

Fig. 6. Three simulated horizon with STL formula satisfaction. The planned trajectory can be seen in red and the actual trajectory is in green, the blue boxes are tubes representations

similar to adaptive convergence techniques used in Monte Carlo methods [23]. The comparison was conducted over the first horizon. The sufficient number of iterations required to reach the stabilization threshold was 1250 for the first-order model and 1500 for the second-order model. With temporal

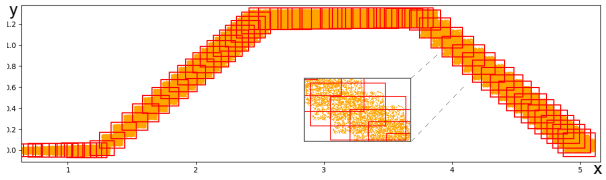

Fig. 7. DynIbex computed tube (red) compared to Monte-Carlo point-cloud (orange), projected onto the x,y plane.

conditions in our monitor, the red boxes in Fig. 7 represent the limit from all trajectories at discrete times for clarity. The DynIbex boxes fully encompass the point cloud generated by the Monte-Carlo method. This is because DynIbex accounts for the worst-case combinations and utilizes a validated numerical integration method. We compare the calculation time on the same laptop (Intel i5-8265u, 8gb RAM) under the same conditions. Results are given in Table II. The Monte-Carlo simulation was performed in Simulink[6] (compiled) using accelerator mode and a simulation profiler to identify and correct flaws in the design. We measured the total execution time for both. Set-based monitoring is significantly faster ($> 10\times$) while providing guarantees, making it well-suited for online monitoring. Given its very fast computation time compared to other methods, the first-order version of the set-based monitor was implemented in the real-world experiment. Moreover, handling complex STL formulas is feasible concerning the computation time of each temporal operator.

[6] https://mathworks.com/

TABLE II

COMPARISON OF TIME EXECUTION BETWEEN SET-BASED METHOD AND MONTE-CARLO METHOD OVER ONE HORIZON (H=4S).

	Set-based monitor	Monte-Carlo monitor
Order 1	0.211s	3.763s
Order 2	3.167s	37.897s

In the presented experiment, there are seven conjunctions of the Globally operator over the simulation time, along with the guaranteed simulation.

VI. CONCLUSION

This work successfully implements a guaranteed real-time monitoring approach for drone trajectory tracking, despite the simplifications and constraints inherent to model testing. The results demonstrate that real-time monitoring can be achieved without precomputing the reachable set, while still providing formal guarantees. The integration of STL formula within the interval analysis framework enables the efficient handling of complex time and state constraints with low computational overhead. This allows for real-time monitoring of various systems, even those with intricate temporal and dynamic behaviors. Additionally, control can be seamlessly incorporated into the forward simulation, provided it can be formulated as an ordinary differential equation and represented as a time-based switched system. Validated numerical simulation also facilitates the analysis of reachability in nonlinear ordinary differential equations.

A key consideration in this approach is that the simulation horizon is constrained by the temporal bounds of the formula. However, unnecessarily extending this horizon may lead to overly conservative outcomes. It is therefore important to balance the required temporal coverage with practical limitations on simulation length. Future work could enable an application to hybrid systems, where branching may occur. This could facilitate the determination of a possible control sequences that lead to failure.

ACKNOWLEDGMENT

The authors acknowledge support from the CIEDS[7] with the STARTS project.

REFERENCES

[1] D. Dvorak and B. Kuipers, "Model-based monitoring of dynamic systems," in *Proceedings of the 11th International Joint Conference on Artificial Intelligence*. Morgan Kaufmann Publishers Inc., 1989, pp. 1238–1243. [Online]. Available: https://ijcai.org/Proceedings/89-2/Papers/068.pdf

[2] E. Bartocci, J. Deshmukh, A. Donzé, G. Fainekos, O. Maler, D. Ničković, and S. Sankaranarayanan, "Specification-based monitoring of cyber-physical systems: A survey on theory, tools and applications," *Lecture Notes in Computer Science*, vol. 10457, pp. 135–175, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-75632-5_5

[3] O. Pettersson, L. Karlsson, and A. Saffiotti, "Model-free execution monitoring in behavior-based robotics," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 37, no. 4, pp. 890–901, 2007. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/4267877

[7] CIEDS: French Interdisciplinary Center for Defense and Security.

[4] A. Abadi, A. E. Amraoui, H. Mekki, and N. Ramdani, "Guaranteed trajectory tracking control based on interval observer for quadrotors," *International Journal of Control*, vol. 93, no. 11, pp. 2602–2611, 2020. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/00207179.2019.1610903

[5] M. Talebberrouane, F. Khan, and Z. Lounis, "Availability analysis of safety critical systems using advanced fault tree and stochastic petri net formalisms," vol. 44, pp. 193–203. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950423016302480

[6] F. Cadini, E. Zio, and D. Avram, "Model-based monte carlo state estimation for condition-based component replacement," vol. 94, no. 3, pp. 752–758. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0951832008002019

[7] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, "Applied interval analysis," L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, Eds. Springer, pp. 11–43.

[8] J.-P. Merlet, "Interval analysis and reliability in robotics," vol. 3, no. 1, pp. 104–130, publisher: Inderscience Publishers. [Online]. Available: https://www.inderscienceonline.com/doi/abs/10.1504/IJRS.2009.026837

[9] S. Largent and J. Alexandre Dit Sandretto, "Trajectory monitoring for a drone using interval analysis," in *Workshop on Planning, Perception and Navigation for Intelligent Vehicles (PPNIV'22)*. Philippe Martinet. [Online]. Available: https://hal.science/hal-04194830

[10] O. Maler and D. Nickovic, "Monitoring temporal properties of continuous signals," in *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, ser. Lecture Notes in Computer Science, vol. 3253. Springer, 2004, pp. 152–166. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-540-30206-3_12

[11] X. Yu, W. Dong, S. Li, and X. Yin, "Model predictive monitoring of dynamical systems for signal temporal logic specifications," *Automatica*, 2023. [Online]. Available: https://doi.org/10.1016/j.automatica.2023.110226

[12] H. Roehm, J. Oehlerking, T. Heinz, and M. Althoff, "Stl model checking of continuous and hybrid systems," in *Automated Technology for Verification and Analysis*, ser. Lecture Notes in Computer Science. Springer International Publishing, 2016, vol. 9938, pp. 412–427.

[13] M. Abate, E. Feron, and S. Coogan, "Monitor-based runtime assurance for temporal logic specifications," *IEEE Transactions on Control Systems Technology*, vol. 29, no. 5, pp. 1932–1947, 2019. [Online]. Available: https://ieeexplore.ieee.org/document/9093858

[14] A. Dokhanchi, B. Hoxha, and G. Fainekos, "On-line monitoring for temporal logic robustness," in *International Conference on Runtime Verification*. Springer, 2014, pp. 231–246. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-11164-3_19

[15] J. Alexandre Dit Sandretto and A. Chapoutot, "Validated explicit and implicit runge-kutta methods," vol. 22. [Online]. Available: https://hal.science/hal-01243053

[16] A. Le Coënt, J. Alexandre dit Sandretto, A. Chapoutot, and L. Fribourg, "An improved algorithm for the control synthesis of nonlinear sampled switched systems," *Formal Methods in System Design*, vol. 53, no. 3, pp. 363–383, 2018. [Online]. Available: https://doi.org/10.1007/s10703-017-0305-8

[17] R. E. Moore, *Interval Analysis*, ser. Series in Automatic Computation. Prentice Hall, 1966.

[18] T. Kapela, M. Mrozek, D. Wilczak, and P. Zgliczyński, "Capd::dynsys: A flexible c++ toolbox for rigorous numerical analysis of dynamical systems," *Communications in Nonlinear Science and Numerical Simulation*, vol. 101, p. 105578, 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1007570420304081

[19] E. Bartocci, Y. Falcone, A. Francalanza, and G. Reger, "Introduction to runtime verification," in *Lectures on Runtime Verification: Introductory and Advanced Topics*. Springer International Publishing, pp. 1–33.

[20] J. Alexandre Dit Sandretto and A. Chapoutot, "Logical differential constraints based on interval boolean tests," in *Fuzzy Techniques: Theory and Applications*. Springer International Publishing, vol. 1000, pp. 788–792.

[21] G. Szederkényi, R. Lakner, and M. Gerzson, *Intelligent Control Systems: An Introduction with Examples*. Springer Science & Business Media.

[22] C. Tang and M. Althoff, "Efficiently obtaining reachset conformance for the formal analysis of robotic contact tasks," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024.

[23] R. Rubinstein and D. Kroese, *Simulation and the Monte Carlo Method*. John Wiley and Sons, Ltd, 2016.